Information Processing Letters 110 (2010) 123–128

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

Efficient algorithms for the all-pairs suffix-prefix problem and the all-pairs substring-prefix problem

Enno Ohlebusch *, Simon Gog

Institute of Theoretical Computer Science, University of Ulm, 89069 Ulm, Germany

ARTICLE INFO

Article history:
Received 16 March 2009
Received in revised form 9 September 2009
Accepted 28 October 2009
Available online 12 November 2009
Communicated by L.A. Hemaspaandra

Keywords:
Algorithms
Suffix array
Suffix-prefix matching

1. Introduction

Gusfield et al. [2,3] devised an optimal $O(n+m^2)$ time algorithm for solving the following problem: Given m strings $S^1, S^2, \ldots, S^m$ of total length n, the *all-pairs suffix-prefix matching problem* is the problem of finding, for all $j \neq k$ with $1 \leqslant j \leqslant m$ and $1 \leqslant k \leqslant m$, the *longest* suffix of S^j which is a prefix of S^k.

Their algorithm uses a generalized suffix tree of the input strings. Suffix trees play a prominent role in algorithmics, but they suffer from two drawbacks:

(i) Although being asymptotically linear, the space consumption of a suffix tree is quite large. Even improved implementations still require 20 bytes per input character in the worst case; see, e.g., [4].
(ii) In most applications, the suffix tree suffers from a poor locality of memory reference, which causes a significant loss of efficiency on cached processor architectures.

In the first part of this paper, we present a new algorithm for solving the above-mentioned problem with the help of a generalized enhanced suffix array, which requires only 8 bytes per input character. Because our algorithm linearly scans the suffix array, it has a good locality behavior. Experimental results show that it uses about 3 times less space and is about 3 times faster than the algorithm using a generalized suffix tree. A further advantage is that our algorithm is easier to implement than the previous one. Although we use ideas from [2] to achieve optimal running time, our algorithm is not a simple simulation of the previous algorithm. (Abouelhoda et al. [1] have shown how algorithms using suffix trees can be simulated on enhanced suffix arrays.) In fact, our algorithm makes use of specific properties of enhanced suffix arrays. In the second part of this paper, we present the first optimal time algorithm for a closely related problem, viz. the all-pairs substring-prefix matching problem.

The main motivation for the all-pairs suffix-prefix matching problem comes from its use in implementing fast approximation algorithms for the *shortest superstring problem*, but there is also a direct application in bioinformatics: In [5] a set of around 1400 expressed sequence tags (ESTs) from the worm *C. elegans* were analyzed for the presence of highly conserved substrings called *ancient conserved regions*, and the analysis was based on the overlap of

* Corresponding author.
E-mail addresses: Enno.Ohlebusch@uni-ulm.de (E. Ohlebusch), Simon.Gog@uni-ulm.de (S. Gog).

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2009.10.015

each pair of ESTs. For details about this application we refer the reader to [3, Section 7.10]. There, one can also find the following hint: "Because there may be some sequencing errors, and because substring containment is possible among strings of unequal length, one should also solve the all-pairs longest common substring problem." It is not difficult to give an $O(m \cdot n)$ time algorithm that solves the *all-pairs longest common substring problem*, but it is unclear whether an optimal $O(n + m^2)$ time algorithm exists.

Here we present the first optimal $O(n + m^2)$ time algorithm that solves a problem that lies in between the two aforementioned problems. This problem is formally defined as follows: Given m strings $S^1, S^2, \ldots, S^m$ of total length n, the *all-pairs substring-prefix matching problem* is the problem of finding, for all $j \neq k$ with $1 \leqslant j \leqslant m$ and $1 \leqslant k \leqslant m$, a *longest* substring of S^j which is a prefix of S^k. Our algorithm scans the generalized enhanced suffix array of the input strings twice and it further uses the method of dynamic programming.

2. Preliminaries

Let S be a string of length n over an ordered alphabet Σ. $S[i]$ denotes the *character at position* i in S, where $1 \leqslant i \leqslant n$. For $i \leqslant j$, $S[i..j]$ denotes the *substring* of S starting with the character at position i and ending with the character at position j. S_i denotes the i-th suffix $S[i..n]$ of S, where $1 \leqslant i \leqslant n$. The *suffix array* SA of the string S is an array of integers in the range 1 to n specifying the lexicographic ordering of the n suffixes of the string S, that is, it satisfies $S_{\mathsf{SA}[1]} < S_{\mathsf{SA}[2]} < \cdots < S_{\mathsf{SA}[n]}$. It can be constructed in linear time and space; see [6] for an overview of suffix array construction algorithms. The *inverse suffix array* SA^{-1} is an array of size n such that for any i with $1 \leqslant i \leqslant n$ the equality $\mathsf{SA}^{-1}[\mathsf{SA}[i]] = i$ holds. Obviously, it can be constructed in linear time from the suffix array. The LCP-array is an array of integers in the range 1 to n such that $\mathsf{LCP}[1] = 0$ and $\mathsf{LCP}[i] = |\mathsf{lcp}(S_{\mathsf{SA}[i-1]}, S_{\mathsf{SA}[i]})|$ for $2 \leqslant i \leqslant n$, where $\mathsf{lcp}(u, v)$ denotes the longest common prefix between two strings u and v. In other words, $\mathsf{LCP}[i]$ is the length of the longest common prefix between the i-th lexicographically smallest suffix of S and its predecessor in the suffix array. The LCP-array can also be constructed in linear time from the suffix array and its inverse; see [7].

Let $S^1, S^2, \ldots, S^m$ be strings of lengths $n_1, n_2, \ldots, n_m$, respectively. The *generalized suffix array* of $S^1, S^2, \ldots, S^m$ is the suffix array SA of the concatenated string $S = S^1 \#_1 S^2 \#_2 \ldots S^m \#_m$, where $\#_1, \#_2, \ldots, \#_m$ are pairwise distinct separator symbols that do not occur in any of the strings (we assume that $\#_1 < \#_2 < \cdots < \#_m$ and all other characters in the alphabet Σ are larger than the separator symbols); see Fig. 1. For a suffix $S_{\mathsf{SA}[i]}$ of S, we denote the prefix of $S_{\mathsf{SA}[i]}$ that ends at the first separator symbol by $S^{\#}_{\mathsf{SA}[i]}$. More precisely, if suffix $S_{\mathsf{SA}[i]}$ starts in string S^j, then the first separator symbol is $\#_j$ and $S^{\#}_{\mathsf{SA}[i]} = S[\mathsf{SA}[i]..j + \sum_{l=1}^{j} n_l]$.

For ease of presentation, we introduce an array str such that $\mathsf{str}[i] = j$ if $S^{\#}_{\mathsf{SA}[i]}$ ends with the character $\#_j$ and an array SA' such that $S^{\#}_{\mathsf{SA}[i]} = S^{j}_{\mathsf{SA}'[i]} \#_j$, where $j = \mathsf{str}[i]$. In other words, a position $\mathsf{SA}[i]$ in the concatenated string S can be determined from the position $\mathsf{SA}'[i]$ in S^j by the equation $\mathsf{SA}[i] = \mathsf{SA}'[i] + j - 1 + \sum_{l=1}^{j-1} n_l$, provided that $j = \mathsf{str}[i]$ is known. Vice versa, the arrays str and SA' can be computed in linear time from the inverse SA^{-1} of the generalized suffix array SA by the pseudo-code in Fig. 2. From another point of view, the two arrays str and SA' specify the lexicographic ordering of all suffixes $\#_1, S^1_1\#_1, \ldots, S^1_{n_1}\#_1, \#_2, S^2_1\#_2, \ldots, S^2_{n_2}\#_2, \ldots, \#_m, S^m_1\#_m, \ldots, S^m_{n_m}\#_m$ of the strings $S^1\#_1, S^2\#_2, \ldots, S^m\#_m$, that is, $S^{\mathsf{str}[i]}_{\mathsf{SA}'[i]}\#_{\mathsf{str}[i]} < S^{\mathsf{str}[i+1]}_{\mathsf{SA}'[i+1]}\#_{\mathsf{str}[i+1]}$ for all i with $1 \leqslant i \leqslant n$, where $n = m + \sum_{j=1}^{m} n_j$; see Fig. 1 for an example. Note that $S^j_p\#_j < S^k_q\#_k$ if and only if either $S^j_p < S^k_q$ or $S^j_p = S^k_q$ and $j < k$.

i	SA	LCP	$S^{\#}_{\mathsf{SA}[i]}$	str	SA'	
1	4	0	$\#_1$	1	4	
2	8	0	$\#_2$	2	4	
3	12	0	$\#_3$	3	4	
4	15	0	$\#_4$	4	3	
5	20	0	$\#_5$	5	5	
6	3	0	$a\#_1$	1	3	
7	14	1	$a\#_4$	4	2	
8	19	1	$a\#_5$	5	4	
9	2	1	$aa\#_1$	1	2	
10	18	2	$aa\#_5$	5	3	
11	1	2	$aaa\#_1$	1	1	i_1
12	17	3	$aaa\#_5$	5	2	
13	5	2	$aac\#_2$	2	1	i_2
14	9	2	$aag\#_3$	3	1	i_3
15	6	1	$ac\#_2$	2	2	
16	10	1	$ag\#_3$	3	2	
17	7	0	$c\#_2$	2	3	
18	11	0	$g\#_3$	3	3	
19	13	1	$ga\#_4$	4	1	i_4
20	16	2	$gaaa\#_5$	5	1	i_5

Fig. 1. The generalized enhanced suffix array of the strings $S^1 = aaa$, $S^2 = aac$, $S^3 = aag$, $S^4 = ga$, and $S^5 = gaaa$. The indices $i_1, i_2, \ldots, i_5$ satisfy $\mathsf{SA}'[i_j] = 1$.

Computation of the arrays str and SA'

```
offset ← 0
for j ← 1 to m do
  for k ← offset + 1 to offset + n_j + 1 do
    str[SA^{-1}[k]] ← j
    SA'[SA^{-1}[k]] ← k − offset
  offset ← offset + n_j + 1
```

Fig. 2. Computation of str and SA' from the inverse SA^{-1} of the generalized suffix array.

The combination of the arrays str and SA' and the LCP-array is called *generalized enhanced suffix array* of $S^1, S^2, \ldots, S^m$ (GESA for short).

3. All-pairs suffix-prefix matching

We start with a high-level description of our solution to the all-pairs suffix-prefix matching problem. The algorithm scans the enhanced suffix array from index $m+1$ (the first m entries can be skipped because they correspond to separator symbols) to index n, keeping track of all suffixes seen so far which are a *prefix* of the current suffix. As in [2], we use m stacks $stack[1], stack[2], \ldots, stack[m]$, one for each string $S^1, S^2, \ldots, S^m$, for this purpose. If the current suffix (at index i) is a prefix of the next suffix (at index $i+1$)

2 (*aa*) 3 (*a*)			2 (*a*)	3 (*aa*) 4 (*a*)
stack[1]	*stack*[2]	*stack*[3]	*stack*[4]	*stack*[5]

Fig. 3. The stacks after index $i = 10$ has been processed. For example, the top element of *stack*[1] is 2 (the corresponding suffix $S^1_2 = aa$ of S^1 is supplied in parentheses).

in the GESA, it is pushed onto *stack*[k], where k is the number of the string S^k to which the current suffix belongs. Fig. 3 shows the stacks after scanning the GESA of Fig. 1 up to index $i = 10$. If the current suffix is a complete string, say $S^k\#_k$, then the top element of *stack*[j], $j \neq k$, is the longest suffix of S^j which is a prefix of S^k, unless there is a suffix of S^j that is identical with S^k and $k < j$. Such an identical suffix, however, must immediately follow $S^k\#_k$ in the GESA; see Lemma 3.1 for details. Continuing our example, the suffix at index $i = 11$ of the GESA in Fig. 1 is the string $S^1 = aaa$. Because the stacks *stack*[2] and *stack*[3] are empty, the empty string is the longest suffix of S^2 and S^3, respectively, which is a prefix of S^1. The suffix *a* corresponding to the topmost element of *stack*[4] is the longest suffix of S^4 that is also a prefix of S^1. By contrast, the suffix *aa* corresponding to the topmost element of *stack*[5] is *not* the longest suffix of S^5 which is a prefix of S^1. This is because the suffix *aaa* of S^5 is also a prefix of S^1. As *aaa* is identical with S^1, it directly follows S^1 in the GESA. Of course, the algorithm must update the stacks appropriately: All suffixes on the stacks that are *not* a prefix of the current suffix must be removed.

We start the detailed description with the following simple lemma.

Lemma 3.1. *In the generalized enhanced suffix array of $S^1, S^2, \ldots, S^m$, let i be an index with $\mathsf{SA}'[i] = 1$ and $\mathsf{str}[i] = k$, i.e., $S^{\#}_{\mathsf{SA}[i]} = S^k\#_k$. If the suffix S^j_p of string S^j is a prefix of string S^k, where $j \neq k$, then either*

- *$S^j_p\#_j$ appears before index i in the GESA of $S^1, S^2, \ldots, S^m$, or*
- *$S^j_p\#_j$ appears at an index q with $i < q \leqslant i + j - k$ in the GESA of $S^1, S^2, \ldots, S^m$ (in this case $k < j$ and all suffixes—without their last character—between i and q are identical with S^k).*

Proof. If S^j_p is a proper prefix of S^k, then (by the definition of the lexicographic ordering) S^j_p is lexicographically smaller than S^k. It is quite obvious that this implies that $S^j_p\#_j$ is lexicographically smaller than $S^k\#_k$. Therefore, it appears before index i. Otherwise, S^j_p is a non-proper prefix of S^k, i.e., $S^j_p = S^k$. We further distinguish two cases.

(1) If $j < k$, then $S^j_p\#_j$ appears before index i because $\#_j < \#_k$, i.e., $S^j_p\#_j$ is lexicographically smaller than $S^k\#_k$.

(2) If $k < j$, then $S^j_p\#_j$ is lexicographically larger than $S^k\#_k$. We observe that only the $n_k + 1$ characters long suffixes of $S^{k+1}\#_{k+1}, S^{k+2}\#_{k+2}, \ldots, S^{j-1}\#_{j-1}$ can possibly lie lexicographically in between $S^k\#_k$ and $S^j_p\#_j$, and an $n_k + 1$ characters long suffix of $S^h\#_h$, where $k < h < j$ and $n_k \leqslant n_h$, lies lexicographically in between $S^k\#_k$ and $S^j_p\#_j$ if and only if the n_k characters long suffix of S^h is identical with S^k. Thus, the lemma follows. □

All-pairs suffix-prefix matching

```
push(lcp_stack, −1)
for i ← m + 1 to n do
  k ← str[i]
  if SA′[i] = 1 then
    for j ← 1 to m do
      if j ≠ k then
        Ov[j, k] ← top(stack[j])
    /* The next five lines handle suffixes identical with S^k */
    if i < n then
      q ← i + 1
      while q ⩽ n and LCP[q] = n_k and |S^{str[q]}_{SA′[q]}| = n_k do
        Ov[str[q], k] ← SA′[q]
        q ← q + 1
  if i < n then
    if LCP[i + 1] < LCP[i] then
      while top(lcp_stack) > LCP[i + 1] do
        for each j in list[top(lcp_stack)] do
          pop(stack[j])
          remove(list[top(lcp_stack)], j)
        pop(lcp_stack)
    else if |S^k_{SA′[i]}| = LCP[i + 1] then
    /* |S^k_{SA′[i]}| = LCP[i + 1] implies that S^k_{SA′[i]} is a prefix of S^{str[i+1]}_{SA′[i+1]} */
      push(stack[k], SA′[i])
      add(list[LCP[i + 1]], k)
      if top(lcp_stack) ≠ LCP[i + 1] then
        push(lcp_stack, LCP[i + 1])
```

Fig. 4. An optimal time algorithm that solves the all-pairs suffix-prefix matching problem.

Fig. 4 shows the algorithm based on the preceding lemma. As already mentioned, it scans the enhanced suffix array from index $m + 1$ to index n, keeping track of all suffixes seen so far which are a *prefix* of the current suffix. To efficiently administer the m stacks *stack*[1], *stack*[2], ..., *stack*[m], the algorithm uses yet another stack *lcp_stack* that contains lcp-values and lists *list*[1], *list*[2], ... Initially, the *lcp_stack* contains the value -1 and all other stacks and lists are empty. In Fig. 4, the procedures *push* and *pop* are the usual stack operations, while *top* provides a pointer to the topmost element of the stack. Furthermore, *add*(*list*, k) appends the element k to the list *list*, while *remove*(*list*, k) removes k from the list *list*. The algorithm stores the solution to the all-pairs suffix-prefix matching problem in an "overlap" matrix *Ov*. The reader is invited to apply the algorithm to the strings of Fig. 1. The resulting overlap matrix *Ov* can be found in Fig. 5.

To prove the correctness of the algorithm, we show that the following properties are invariants of the outer for-loop: Before the for-loop is executed for some i with $m + 1 \leqslant i \leqslant n$, the stacks *stack*[1], *stack*[2], ..., *stack*[m] contain exactly the suffixes seen so far which are a prefix of $S^{\mathsf{str}[i]}_{\mathsf{SA}'[i]}$. (As a matter of fact, the stacks do not contain suffixes but suffix numbers. For example, if $p = top(stack[j])$, then the suffix number p corresponds to the suffix S^j_p of S^j.) Furthermore, the lcp-values on *lcp_stack* are strictly increasing and *list*[ℓ] is non-empty if and only if ℓ is an element of *lcp_stack*. Moreover, j is in *list*[ℓ] if and only if *stack*[j]

Ov	1	2	3	4	5
1	⊥	2 (*aa*)	2 (*aa*)	⊥	⊥
2	⊥	⊥	⊥	⊥	⊥
3	⊥	⊥	⊥	3 (*g*)	3 (*g*)
4	2 (*a*)	2 (*a*)	2 (*a*)	⊥	1 (*ga*)
5	2 (*aaa*)	3 (*aa*)	3 (*aa*)	⊥	⊥

Fig. 5. The overlap matrix *Ov* after the algorithm in Fig. 4 was applied to the strings of Fig. 1. For example, $Ov[4,1]=2$ means that $S_2^4 = a$ is the longest suffix of $S^4 = ga$ that matches a prefix of $S^1 = aaa$. Furthermore, ⊥ stands for an undefined value.

contains the suffix $S^j_{n_j-\ell+1}$ (the suffix of S^j which has length ℓ).

Under the assumption that the above-mentioned properties hold before the for-loop is executed for some i, we show that the properties also hold before the for-loop is executed for $i+1$. We proceed by case analysis.

If $\mathsf{LCP}[i+1] \geqslant \mathsf{LCP}[i]$, then all suffixes seen so far which are a prefix of $S^{\mathsf{str}[i]}_{\mathsf{SA}'[i]}$ are also a prefix of $S^{\mathsf{str}[i+1]}_{\mathsf{SA}'[i+1]}$. Note that the longest suffix-prefix match seen so far cannot have a length strictly greater than $\mathsf{LCP}[i]$, i.e., $top(lcp\_stack) \leqslant \mathsf{LCP}[i]$. Furthermore, the else-if-condition tests whether $S^k_{\mathsf{SA}'[i]}$ (where $k = \mathsf{str}[i]$) is a prefix of $S^{\mathsf{str}[i+1]}_{\mathsf{SA}'[i+1]}$. If this is the case, the suffix number $\mathsf{SA}'[i]$ representing the suffix $S^k_{\mathsf{SA}'[i]}$ is pushed onto $stack[k]$ and the string number k is added to $list[|S^k_{\mathsf{SA}'[i]}|]$. Moreover, if $top(lcp\_stack) \neq \mathsf{LCP}[i+1]$, then the lcp-value $\mathsf{LCP}[i+1]$ is pushed onto lcp_stack. Note that $top(lcp\_stack) \neq \mathsf{LCP}[i+1]$ implies $top(lcp\_stack) < \mathsf{LCP}[i+1]$ because $top(lcp\_stack) \leqslant \mathsf{LCP}[i] \leqslant \mathsf{LCP}[i+1]$. Therefore, the properties also hold before the for-loop is executed for $i+1$.

Otherwise, if $\mathsf{LCP}[i+1] < \mathsf{LCP}[i]$, then all suffixes of length ℓ with $\mathsf{LCP}[i] \geqslant \ell \geqslant \mathsf{LCP}[i+1]+1$ must be removed from the stacks because these suffixes are not a prefix of $S^{\mathsf{str}[i+1]}_{\mathsf{SA}'[i+1]}$. For each lcp-value ℓ on the stack lcp_stack (from top to bottom, i.e., in decreasing order), we use the list $list[\ell]$ to find all stacks that have a suffix of length ℓ on top and successively remove these top elements from the stacks and the string numbers from $list[\ell]$ until $list[\ell]$ is empty. When $list[\ell]$ is empty, we pop ℓ from the stack lcp_stack. Again, the properties also hold before the for-loop is executed for $i+1$. (Observe that in this case $S^k_{\mathsf{SA}'[i]}$ cannot be a prefix of $S^{\mathsf{str}[i+1]}_{\mathsf{SA}'[i+1]}$ because $|S^k_{\mathsf{SA}'[i]}| \geqslant \mathsf{LCP}[i] > \mathsf{LCP}[i+1]$.)

The correctness of the algorithm in Fig. 4 now follows from the invariant properties. If the for-loop is executed for some i with $\mathsf{SA}'[i] = 1$ and $\mathsf{str}[i] = k$, then the topmost element of $stack[j]$, say $t = top(stack[j])$, represents the longest suffix S^j_t of S^j seen so far which is a prefix of S^k. Lemma 3.1 implies that S^j_t is the longest suffix of S^j which is a prefix of S^k, unless $k < j$ and there is a suffix S^j_p of S^j such that $S^j_p = S^k$. However, all suffixes that are identical with S^k directly (precede or) succeed S^k in the suffix array and the algorithm deals with this special case.

Let us analyze the worst-case time complexity of the algorithm in Fig. 4. Each of the n suffixes is pushed at most once onto a stack. This implies that there are at most n elements added to the lists. Consequently, there are at most $2n$ push and pop operations on the stacks, and at most $2n$ add and remove operations on the lists. There are m indices $i_1, i_2, \ldots, i_m$ such that $\mathsf{SA}'[i_j] = 1$. For every i_j, the inner for-loop is executed m times. Furthermore, there are at most $m-1$ suffixes which are identical with the current suffix S^k (excluding S^k itself). Thus, for every i_j the algorithm takes $O(m)$ time. This sums up to $O(m^2)$ time for all m indices $i_1, i_2, \ldots, i_m$. Altogether, the worst-case time complexity of the algorithm in Fig. 4 is $O(n+m^2)$. This is optimal because n is the size of the input and m^2 is the size of the output.

4. All-pairs substring-prefix matching

In this section, we present two algorithms that solve the all-pairs substring-prefix matching problem. Here we confine ourselves to finding the *length* of a longest substring-prefix match of S^j and S^k, i.e., we compute

$$R[j,k] = \max\big\{\ell\colon\ S^j[i..i+\ell-1] = S^k[1..\ell] \quad \text{where } 1 \leqslant i \leqslant n_j\big\}$$

for every $j \neq k$ with $1 \leqslant j \leqslant m$ and $1 \leqslant k \leqslant m$, where $n_j = |S^j|$. It is easy, however, to modify our algorithms so that they also output a position i in S^j which attains the maximum $R[j,k]$.

We build the generalized suffix array of the strings $S^1, S^2, \ldots, S^m$, which has $n = m + \sum_{l=1}^{m} n_l$ rows. Let $i_1, i_2, \ldots, i_m$ be the indices in the GESA such that $S^{\#}_{\mathsf{SA}[i_k]} = S^k\#_k$, i.e., $\mathsf{str}[i_k] = k$ and $\mathsf{SA}'[i_k] = 1$. W.l.o.g. we may assume that $i_1 < i_2 < \cdots < i_m$ (if this is not the case, we just renumber the strings). In the example of Fig. 1, we have $i_1 = 11$, $i_2 = 13$, $i_3 = 14$, $i_4 = 19$, and $i_5 = 20$. Obviously, the indices $i_1, i_2, \ldots, i_m$ can be computed in linear time.

To find the longest substring-prefix match of strings S^j and S^k, we consider index i_k. Let p be the largest index with $p < i_k$ such that $\mathsf{str}[p] = j$, and let q be the smallest index with $i_k < q$ such that $\mathsf{str}[q] = j$. In contrast to index p such an index q may not exist. If such an index q exists, then the length of the longest substring-prefix match of S^j and S^k is

$$\max\Big\{\big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[p]}, S^{\#}_{\mathsf{SA}[i_k]}\big)\big|, \big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[q]}\big)\big|\Big\}$$

because $|\mathsf{lcp}(S^{\#}_{\mathsf{SA}[p']}, S^{\#}_{\mathsf{SA}[i_k]})| \leqslant |\mathsf{lcp}(S^{\#}_{\mathsf{SA}[p]}, S^{\#}_{\mathsf{SA}[i_k]})|$ for all $p' < p$ and similarly $|\mathsf{lcp}(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[q']})| \leqslant |\mathsf{lcp}(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[q]})|$ for all $q' > q$. If such an index q does not exist, then the length of the longest substring-prefix match of S^j and S^k is $|\mathsf{lcp}(S^{\#}_{\mathsf{SA}[p]}, S^{\#}_{\mathsf{SA}[i_k]})|$.

Now a simple algorithm for solving the all-pairs substring-prefix matching problem is based on the obvious equations

$$\begin{aligned}&\big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[p]}, S^{\#}_{\mathsf{SA}[i_k]}\big)\big| \\ &\quad = \min\big\{\mathsf{LCP}[p+1], \mathsf{LCP}[p+2], \ldots, \mathsf{LCP}[i_k]\big\} \\ &\big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[q]}\big)\big| \\ &\quad = \min\big\{\mathsf{LCP}[i_k+1], \mathsf{LCP}[i_k+2], \ldots, \mathsf{LCP}[q]\big\}\end{aligned}$$

For each k, $1 \leqslant k \leqslant m$, the algorithm starts at index i_k and linearly scans the LCP-array downwards, keeping track of

Computation of the D-matrix

```
procedure ComputeMatrix
for j ← 1 to m do
    for k ← 1 to m do
        D[j,k] ← ⊥
for k ← 1 to m do
    lcp ← ∞
    for i ← i_k + 1 to i_{k+1} do
        j ← str[i]
        lcp ← min{lcp, LCP[i]}
        if j ≠ k ∧ D[j,k] = ⊥ then
            D[j,k] ← lcp
    if k ≠ m then
        D[k,k] ← lcp
```

```
procedure CompleteMatrix
for j ← 1 to m − 1 do
    if D[j,m] = ⊥ then
        D[j,m] ← 0
for k ← m − 1 downto 1 do
    for j ← k + 2 to m do
        if D[j,k] = ⊥ then
            D[j,k] ← min{D[k,k], D[j,k+1]}
    for j ← k − 1 downto 1 do
        if D[j,k] = ⊥ then
            D[j,k] ← min{D[k,k], D[j,k+1]}
```

Fig. 6. Procedure *ComputeMatrix* computes the partial D-matrix, while procedure *CompleteMatrix* completes the partial D-matrix by dynamic programming.

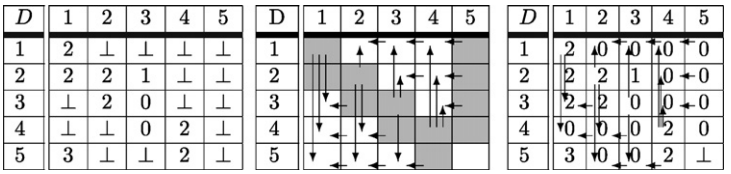

Fig. 7. Left: The partial D-matrix delivered by the procedure *ComputeMatrix*. Middle: The dynamic programming recurrences. Right: The complete D-matrix obtained by the procedure *CompleteMatrix*.

the current lcp-value. Whenever it encounters a string j that has not been seen yet, it stores the length of the longest "downwards" substring-prefix match of S^j and S^k as entry $D[j,k]$ in an $m \times m$ matrix D. That is, it computes

$$D[j,k] = \max\{\ell\colon\ S^j[i..i+\ell-1] = S^k[1..\ell] \text{ where } 1 \leqslant i \leqslant n_j \text{ and } S^j_i > S^k\}$$

Similarly, starting at index i_k, the algorithm linearly scans the LCP-array upwards and stores the length of the longest "upwards" substring-prefix match of S^j and S^k as entry $U[j,k]$ in an $m \times m$ matrix U. That is, it computes

$$U[j,k] = \max\{\ell\colon\ S^j[i..i+\ell-1] = S^k[1..\ell] \text{ where } 1 \leqslant i \leqslant n_j \text{ and } S^j_i < S^k\}$$

Finally, it determines $R[j,k]$, the length of the longest substring-prefix match of S^j and S^k, by taking the maximum of the values $U[j,k]$ and $D[j,k]$. A drawback of this algorithm is its time complexity $O(mn+m^2)$. However, a slight modification will yield an $O(n+m^2)$ time algorithm as we shall see next. This worst-case time complexity is optimal because n is the size of the input and m^2 is the size of the output. We focus solely on the computation of the D-matrix because the computation of the U-matrix is very similar.

After matrix initialization, procedure *ComputeMatrix* from Fig. 6 starts at index i_1 and scans the LCP-array downwards, computing entries of the first column of the D-matrix. It stops filling the first column of the D-matrix when it encounters the index i_2 (this means that the column may only be partially filled). At that point the current lcp-value equals $|\mathsf{lcp}(S^1, S^2)|$, and this value is stored in $D(1,1)$. Then it continues the downward scan, but now it computes entries of the second column of the D-matrix, etc. The algorithm also works for the last column if we define $i_{m+1} = m$. It is clear from the preceding considerations that if a value is assigned to $D[j,k]$, where $j \neq k$, then this assignment is correct. Moreover, note that for each k with $1 \leqslant k \leqslant m-1$, $D[k,k]$ stores the value $|\mathsf{lcp}(S^k, S^{k+1})|$. The left-hand side of Fig. 7 shows the D-matrix after the application of procedure *ComputeMatrix* to the strings of Fig. 1. Those cells in the D-matrix which are definitely filled are the shaded cells on the diagonals of the middle matrix of Fig. 7.

The full D-matrix can easily be obtained by a simple dynamic programming algorithm, as detailed in procedure *CompleteMatrix* of Fig. 6. The dynamic programming recurrences are illustrated in the middle of Fig. 7: The last column can be filled (that is why the cells in the last column of the middle matrix are also shaded) and a non-shaded cell that is still empty can be filled by the minimum of the two values at the beginning of the incoming arrows. The right-hand side of Fig. 7 shows the completed D-matrix of our example.

It remains to show the correctness of the procedure *CompleteMatrix*. In the first for-loop of the procedure the last column of the D-matrix is filled: If $D[j,m] = \perp$, then the value of $D[j,m]$ is set to 0. This is a correct assignment because $D[j,m] = \perp$ means that there is no index i with $i_m < i \leqslant n$ and $\mathsf{str}[i] = j$. In the second for-loop of the procedure, consider k and j with $1 \leqslant k \leqslant m-1$ and either $k+2 \leqslant j \leqslant m$ or $k-1 \geqslant j \geqslant 1$. If $D[j,k] = \perp$, then the value of $D[j,k]$ is set to $\min\{D[k,k], D[j,k+1]\}$. This is also a correct assignment because $D[j,k] = \perp$ implies that there is no index i with $i_k < i \leqslant i_{k+1}$ and $\mathsf{str}[i] = j$, and we infer

$$\begin{aligned} D[j,k] &= \max\big(\big\{\big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[i]}\big)\big|\colon\ i_k < i \leqslant n \text{ and } \mathsf{str}[i] = j\big\} \cup \{0\}\big) \\ &= \max\big(\big\{\big|\mathsf{lcp}\big(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[i]}\big)\big|\colon\ i_{k+1} < i \leqslant n \text{ and } \mathsf{str}[i] = j\big\} \cup \{0\}\big) \end{aligned}$$

$$= \min\Big\{\big|\mathrm{lcp}\big(S^{\#}_{\mathsf{SA}[i_k]}, S^{\#}_{\mathsf{SA}[i_{k+1}]}\big)\big|,$$
$$\max\Big(\Big\{\big|\mathrm{lcp}\big(S^{\#}_{\mathsf{SA}[i_{k+1}]}, S^{\#}_{\mathsf{SA}[i]}\big)\big| : i_{k+1} < i \leqslant n \text{ and } \mathsf{str}[i] = j\Big\} \cup \{0\}\Big)\Big\}$$
$$= \min\big\{D[k,k], D[j,k+1]\big\}$$

5. Experimental results

A C++ implementation of the new all-pairs suffix-prefix matching algorithm can be found at http://www.uni-ulm.de/in/theo/research/seqana.html. We applied the program to the complete EST database of *C. elegans* at NCBI (http://ncbi.nlm.nih.gov/, visited September 7, 2009). The database contained $m = 334,465$ ESTs (strings over the DNA alphabet) and the total length of the concatenated strings was $n = 167,369,577$. Because the run time of the algorithm depends quadratically on the output size, we limited the output to suffix-prefix matches of length $\geqslant 10$. The execution of the program took 70 minutes and it used 2.5 GB main memory (including the output) on the following system: AMD Opteron 1222 (3 GHz), cache size 1 MB, main memory 4 GB, GCC version 4.2.1.

Since we could not find an implementation of the algorithm described in Gusfield et al. [2,3], we implemented it ourselves. As a basis, we used the core suffix tree library of the MUMmer3 package (http://mummer.sourceforge.net/), implemented by Stefan Kurtz [4]. This library is widely accepted as one of the best suffix tree implementations worldwide. However, because of a limitation on the alphabet size (i.e., the number of separator symbols), we could run the suffix tree algorithm only for $m \leqslant 200$ strings. For 200 ESTs ($n = 107,139$), it used 4 MB of main memory while the new suffix array algorithm used only 1.3 MB. Moreover, the new suffix array algorithm runs about 3 times faster than the suffix tree algorithm (1.47 seconds vs. 4.74 seconds for 100 executions of the programs).

References

[1] M.I. Abouelhoda, S. Kurtz, E. Ohlebusch, Replacing suffix trees with enhanced suffix arrays, J. Discrete Algorithms 2 (2004) 53–86.
[2] D. Gusfield, G. Landau, B. Schieber, An efficient algorithm for the all pairs suffix-prefix problem, Inform. Process. Lett. 41 (4) (1992) 181–185.
[3] D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press, New York, 1997.
[4] S. Kurtz, Reducing the space requirement of suffix trees, Software—Practice and Experience 29 (13) (1999) 1149–1171.
[5] P. Green, D. Lipman, D. Hillier, R. Waterston, D. States, J. Claverie, Ancient conserved regions in new gene sequences and the protein databases, Science 259 (1993) 1711–1716.
[6] S. Puglisi, W. Smyth, A. Turpin, A taxonomy of suffix array construction algorithms, ACM Computing Surveys 39 (2) (2007) 1–31.
[7] T. Kasai, G. Lee, H. Arimura, S. Arikawa, K. Park, Linear-time longest-common-prefix computation in suffix arrays and its applications, in: Proc. 12th Symposium on Combinatorial Pattern Matching, in: Lecture Notes in Comput. Sci., vol. 2089, Springer-Verlag, Berlin, 2001, pp. 181–192.